

Laporan Tugas Kecil 2

IF2211 Strategi Algoritma

Semester II Tahun 2025/2026



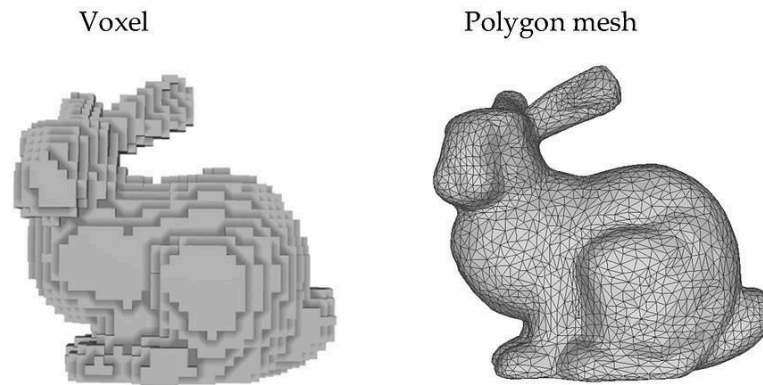
Disusun oleh:

Mikhael Andrian Yonatan / 13524051

Nicholas Luis Chandra / 13524105

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

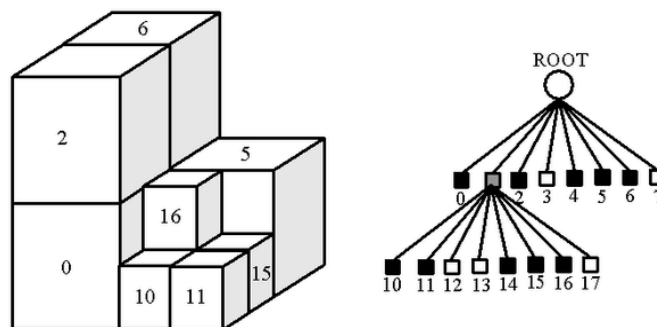
Deskripsi Persoalan



Visualisasi *voxelization*

Sumber: <https://www.mdpi.com/1424-8220/22/18/7040>

Voxelization merupakan proses konversi objek tiga dimensi yang umum menjadi sebuah model yang tersusun dari kubus-kubus kecil seperti di Minecraft. Proses konversi ini dilakukan dengan memanfaatkan struktur data Octree, yaitu sebuah tree yang setiap percabangan terdiri dari 8 anak. Melalui Octree, sebuah ruang kubus 3D dapat dipotong menjadi 8 kubus berbeda, yang masing-masing dapat dipotong lagi menjadi 8 kubus yang lebih kecil.



Visualisasi Octree terhadap 3D Model

Sumber:

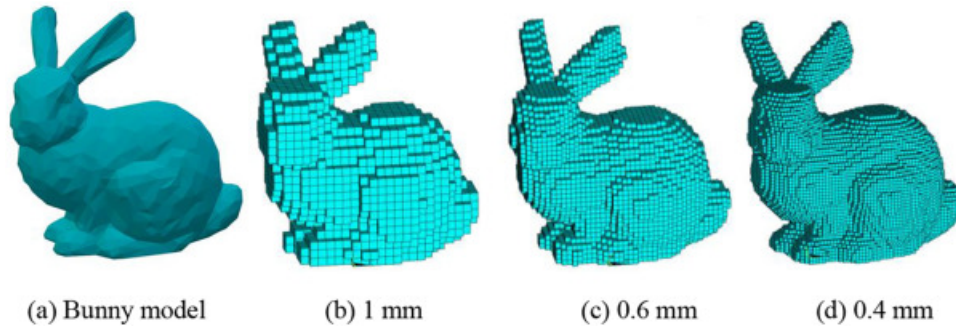
https://www.researchgate.net/figure/a-Octants-b-Octree-Structure-c-Linear-Octree-Each-of-the-cubes-has-a-locational-code_fig1_221115507

Tugas Anda adalah mengimplementasikan sebuah program yang dapat mengkonversi model .obj yang umum menjadi model .obj yang terdiri dari voxel dengan memanfaatkan properti Octree. Program melakukan konversi model dengan menggunakan algoritma divide and conquer.

Dasar Teori

Voxelization

Voxelization adalah proses mengubah permukaan objek 3D yang kontinu (terdiri dari vertex dan segitiga/ faces) menjadi format diskrit berupa kumpulan kubus-kubus kecil yang disebut voxel.



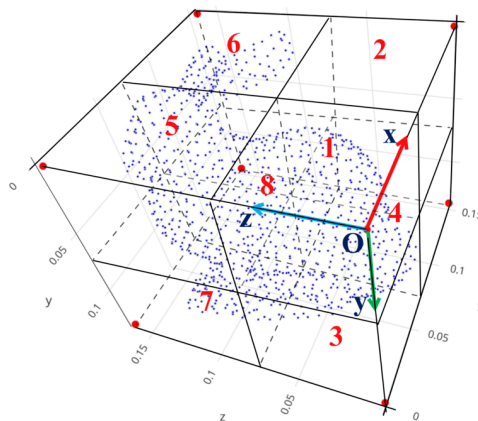
Ilustrasi voxelization

Sumber :

<https://www.google.com/url?sa=t&source=web&rct=j&url=https%3A%2F%2Fwww.sciencedirect.com%2Fscience%2Farticle%2Fpii%2FS0264127519308470&opi=89978449>

Bounding Box AABB (Axis-Aligned Bounding Box)

Sebelum program dapat melakukan voxelization, program harus mengetahui seberapa besar ruang keseluruhan yang ditempati oleh model 3D tersebut. Hal tersebut dilakukan dengan mencari nilai X, Y, dan Z minimum dan maksimum dari seluruh vertex yang dibaca dari file .obj. Ruang awal inilah yang disebut *Axis-Aligned Bounding Box* (AABB). Jika model aslinya panjang, maka AABB awalnya akan berbentuk persegi panjang. Sebelum masuk ke algoritma utama, AABB ini wajib diubah menjadi kubus yang mengelilingi persegi panjang tersebut. Jika tidak diubah menjadi kubus, saat dibagi-bagi nanti, voxel yang dihasilkan akan berbentuk persegi panjang pipih (tidak *uniform*).



Ilustrasi Bounding-Box AABB

Sumber :

https://www.google.com/url?sa=t&source=web&rct=j&url=https%3A%2F%2Fwww.researchgate.net%2Ffigure%2FA-demonstration-of-Axis-Aligned-Bounding-Box-AABB-divided-into-8-sub-cubes_fig5_335088056&opi=89978449

Algoritma Divide & Conquer dengan Octree

Divide and Conquer adalah sebuah paradigma desain algoritma yang memecahkan masalah besar dan rumit dengan membaginya menjadi masalah-masalah kecil yang lebih sederhana dan sejenis. Setelah masalah-masalah kecil ini diselesaikan, hasilnya digabungkan kembali untuk membentuk solusi dari masalah besar yang asli,

Dalam kasus program yang akan diimplementasikan, implementasi paradigma *divide & conquer* adalah sebagai berikut :

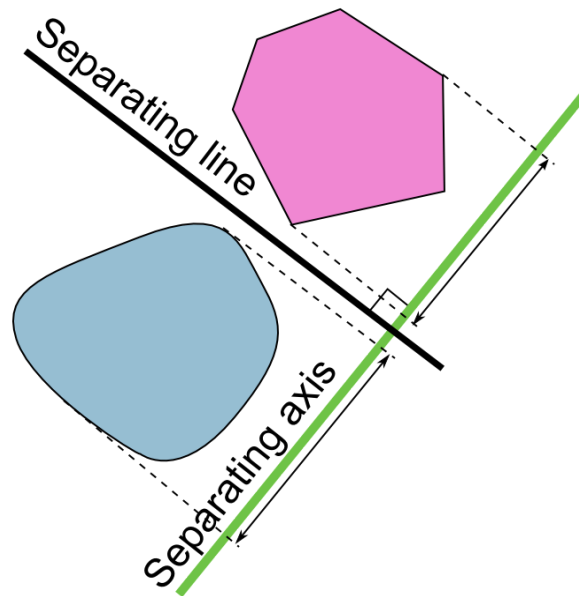
- *Divide*: Ambil kubus AABB utama, lalu potong titik tengah sumbu X, Y, dan Z untuk membaginya menjadi 8 sub-kubus (oktan) yang ukurannya persis sama.
- *Conquer*: Secara rekursif, periksa masing-masing dari 8 oktan tersebut.
 - Apakah oktan ini kosong (tidak menyentuh segitiga manapun dari model asli)? Jika ya, buang oktan tersebut (pruning dini).
 - Apakah oktan ini menyentuh segitiga model? Jika ya, potong lagi oktan ini menjadi 8 bagian lebih kecil, dan ulangi prosesnya sampai mencapai depth maksimum.

Separating Axis Theorem (SAT)

Separating Axis Theorem (SAT) adalah sebuah metode matematis yang digunakan untuk mendeteksi perpotongan (interseksi) antara dua objek cembung (*convex*), yang dalam kasus ini adalah antara kotak oktan (AABB) dan permukaan segitiga dari model 3D. Prinsip dasar dari teorema ini adalah untuk memeriksa apakah dua objek tidak saling bersinggungan dengan memeriksa apakah terdapat setidaknya satu sumbu (axis) di mana proyeksi bayangan kedua objek tersebut benar-benar terpisah. Sebaliknya, untuk membuktikan bahwa kedua objek tersebut pasti berpotongan, harus dipastikan bahwa tidak ada satupun sumbu pemisah di antara keduanya (semua proyeksi pada sumbu uji saling tumpang tindih / overlap).

Dalam program ini, SAT diaplikasikan pada saat proses penelusuran Octree untuk menentukan apakah sebuah oktan menyentuh segitiga model atau kosong. Pengujian ini dilakukan dengan memanfaatkan konsep *dot product* untuk memproyeksikan masing-masing vertex dari segitiga serta titik/dimensi dari kotak (oktan) ke sebuah sumbu uji. Jika rentang nilai proyeksi kedua objek tersebut pada sumbu uji saling tumpang tindih (*overlap*), maka pengecekan dilanjutkan. Sumbu yang diuji totalnya berjumlah 13 sumbu kandidat, dengan rincian sebagai berikut:

- 3 sumbu normal dari permukaan kotak (searah dengan sumbu X, Y, dan Z).
- 1 sumbu normal dari permukaan segitiga.
- 9 sumbu yang didapat dari hasil perkalian silang (*cross product*) antara 3 tepi kotak dan 3 tepi segitiga.



Ilustrasi *Separating Axis Theorem* (SAT)

Sumber :

https://www.google.com/url?sa=t&source=web&rct=j&url=https%3A%2F%2Fen.wikipedia.org%2Fwiki%2FHyperplane_separation_theorem&opi=89978449

Implementasi Program

a. Struktur Direktori

```
Tucil2_13524051_13524105/  
|__ bin/  
| |__ voxelizer.exe  
|__ doc/  
| |__ Laporan Laporan Tucil 2 - 13524103 - 13524105.pdf  
|__ src/  
| |__ file/  
| | |__ file.go  
| |__ geometry/  
| | |__ geometry.go  
| |__ models/  
| | |__ models.go  
| |__ voxelization/  
| | |__ voxelization.go  
| |__ main.go  
|__ test/  
|__ go.mod  
|__ README.md
```

Secara garis besar,

- Folder src berisi source code program dalam bahasa Go.
- Folder bin berisi executable file hasil kompilasi untuk menjalankan program.
- Folder test berisi file data input serta data output (solusi) yang dihasilkan oleh program.
- Folder doc berisi laporan tugas kecil dalam bentuk dokumen PDF.
- File go.mod berisi konfigurasi untuk mendeklarasikan jalur impor (root) dan melacak dependensi eksternal pada proyek Go.

b. Algoritma

Proses voxelization bekerja selama belum mencapai batas kedalaman maksimum yang telah didefinisikan. Algoritma divide and conquer akan terus dilakukan selama $currentDepth < maxDepth$, dengan algoritmanya sebagai berikut :

1. Buat kubus pertama, yaitu kubus yang dapat membungkus seluruh mesh model.
2. Periksa face valid yang ada di dalam kubus untuk diwarisi dan dicek oleh sub kubus (anak). Jika daftar $face > 0$ dan $currentDepth < maxDepth$, maka bagi jadi 8 sub kubus.

3. Lanjut ke sub kubus. Jika daftar face > 0 dan currentDepth < maxDepth, maka bagi jadi 8 subkubus lagi. Jika tidak (daftar face = 0), maka hentikan penelusuran lebih dalam pada kubus ini dan abaikan.
4. Jika sudah mencapai kedalaman currentDepth = maxDepth, maka berhenti dan catat informasi kubus.
5. Lakukan langkah 2, 3, dan 4 secara rekursif ke setiap kubus.

Algoritma *divide and conquer* diterapkan secara rekursif ke setiap kubus, hal ini dapat dilihat pada cara kerja algoritma yang terus mendekomposisi kubus menjadi 8 sub kubus dan ruang pengecekan *face valid* yang terus diperkecil juga. Hingga akhirnya, akan diperoleh kubus-kubus *valid* (mengandung *face*) pada kedalaman maksimum saja.

c. Implementasi Kode Program

1. models.go

File ini mengandung seluruh *struct* yang akan digunakan dalam program

Vertex mendefinisikan simpul (titik)

```
type Vertex struct {
    X, Y, Z float64
}
```

Face mendefinisikan *polygon* (permukaan) yang terbentuk dari Vertex

```
type Face struct {
    V1, V2, V3 int
}
```

Cube mendefinisikan kubus

```
type Cube struct {
    Center Vertex
    Size    float64
}
```

OctreeNode mendefinisikan struktur data Octree dan informasi lainnya yang dibutuhkan

```
type OctreeNode struct {
    CurrentCube  Cube
    Children [8]*OctreeNode
    Face []int
    IsLeaf  bool
    IsVoxel bool
}
```

OctreeStats mendefinisikan informasi yang diperlukan untuk output

```
type OctreeStats struct {
    NodesByDepth []int
}
```

```

        SkippedByDepth []int
        MaxDepth       int
        LeafNodes      []*OctreeNode
    }

    Model mendefinisikan properti dari mesh model
    type Model struct {
        Vertices []Vertex
        Faces    []Face
    }

```

2. main.go

File ini mengandung main sebagai integrasi dari modul-modul yang telah diimplementasikan

```

func main() {
    CARA KERJA:
    1. Masukkan nama file .obj
    2. Masukkan kedalaman maksimum
    3. Menjalankan StartVoxelization
    4. Menampilkan statistik hasil dan menghasilkan file .obj
}

```

3. voxelization.go

File ini mengandung algoritma dari proses voxelization secara divide and conquer

Memulai voxelization dengan membuat kubus pertama

```

func StartVoxelization(parsedModel *models.Model, maxDepth int)
(*models.OctreeNode, *models.OctreeStats) {
    INPUT : Properti model awal, kedalaman maksimum
    OUTPUT: OctreeNode dan stats
    CARA KERJA:
    1. Cari nilai ekstrem pada setiap sumbu
    2. Pilih sumbu dengan selisih terbesar sebagai ukuran kubus
       pembungkus
    3. Isi daftar face yang valid (karena kubus pertama, maka
       seluruh face dari model)
    4. Deklarasi stats
    5. Terapkan divideConquer
}

```

Memproses kubus secara rekursif dan membangun Octree

```

func divideConquer(cube models.Cube, parsedModel *models.Model,

```



```
parentFaces []int, currentDepth, maxDepth int, stats
*models.OctreeStats) *models.OctreeNode {
    INPUT : Kubus yang diproses, properti model, daftar face parent,
            currentDepth, maxDepth, dan stats)
    OUTPUT: OctreeNode
    CARA KERJA:
    1. Cek face yang valid dan masukkan ke daftar
    2. Jika tidak ada face valid, maka abaikan kubus ini
    3. Jika currentDepth = maxDepth, maka hentikan
    4. Jika currentDepth < maxDepth, maka buat 8 subkubus
    5. Terapkan divideConquer lagi ke masing-masing subkubus
}
```

4. geometry.go

File ini mengandung algoritma pengecekan kubus valid (terdapat face)

Mengecek face yang valid pada kubus berdasarkan daftar face valid dari parent

```
func GetIntersectingFaces(cube models.Cube, vertices
[]models.Vertex, faces []models.Face, parentFaceIndices []int) []int
{
    INPUT : Kubus yang diproses, properti model, daftar face parent)
    OUTPUT: Daftar face yang valid di kubus ini
    CARA KERJA:
    1. Untuk setiap face di daftar face valid milik parent lakukan 3 tes
    2. Tes1: Bungkus face dengan balok, cek apakah kubus membungkus atau dibungkus balok
    3. Tes2: Perluas face sebagai bidang, cek apakah kubus berpotongan dengan bidang
    4. Tes3: Cek apakah kubus benar-benar berpotongan dengan face (bukan bidang hasil perluasan)
    5. Face valid, apabila lulus ketiga tes yang ada
}
```

5. file.go

File ini mengandung fungsi untuk membaca dan menulis file

Membaca file .obj

```
func ParseOBJ(inputPath string) (*models.Model, error) {
    INPUT : File path
    OUTPUT: Properti model
```

CARA KERJA:

1. Baca file baris per baris dan lakukan validasi sesuai dengan aturan yang sudah didefinisikan

}

Menulis file .obj

```
func WriteObj(filename string, leaves []*models.OctreeNode) error {
```

INPUT : File path dan daftar kubus valid

OUTPUT: File .obj

CARA KERJA:

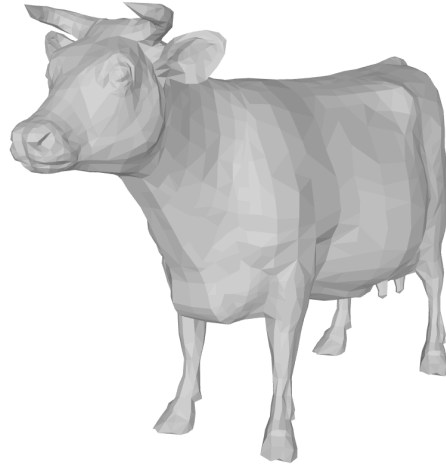
1. Baca daftar kubus valid
2. Konversi tiap kubus menjadi 8 vertecies dan 12 faces
3. Tulis ke output file .obj

}

Pengujian Program

Pengujian dilakukan dengan menggunakan <https://3dviewer.net>

1. cow.obj



Input :

Masukan nama file yang ada di /test (misal : cow.obj) : cow.obj
Masukan max depth voxelization Notes : Semakin besar semakin lama waktu menunggu: 6

Output:

Mulai memproses file: test/cow.obj
Target kedalaman Octree: 6

=== Statistik Voxelization ===

Banyaknya voxel yang terbentuk: 5627
Banyaknya vertex yang terbentuk: 45016
Banyaknya faces yang terbentuk: 67524
Statistik node octree yang terbentuk

1 : 8

2 : 64

3 : 160

4 : 704

5 : 2656

6 : 10896

Statistik node yang tidak perlu ditelusuri

1 : 0

2 : 44

3 : 72

4 : 372

5 : 1294

6 : 0

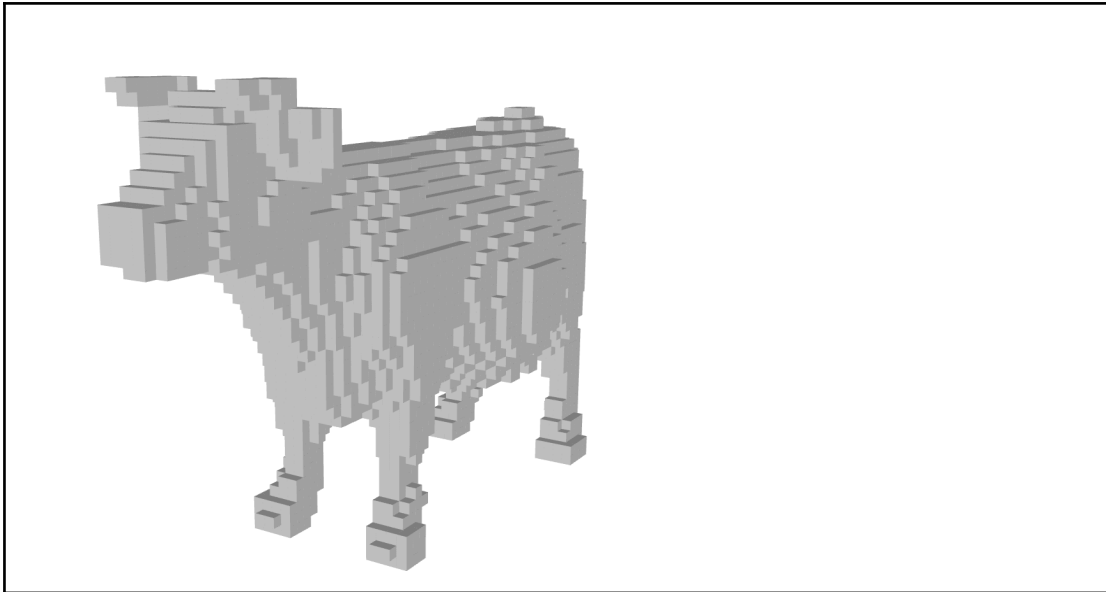
Kedalaman octree: 6

Lama waktu program berjalan: 107.486196ms

Path dimana file .obj

disimpan:

test/cow-Voxelization_Result_Depth-6.obj



Input:

Masukan nama file yang ada di /test (misal : cow.obj) : cow.obj
Masukan max depth voxelization Notes : Semakin besar semakin lama waktu menunggu: 8

Output:

Mulai memproses file: test/cow.obj
Target kedalaman Octree: 8

=== Statistik Voxelization ===

Banyaknya voxel yang terbentuk: 92099
Banyaknya vertex yang terbentuk: 736792
Banyaknya faces yang terbentuk: 1105188
Statistik node octree yang terbentuk

1 : 8
2 : 64
3 : 160
4 : 704
5 : 2656
6 : 10896
7 : 45016
8 : 182912

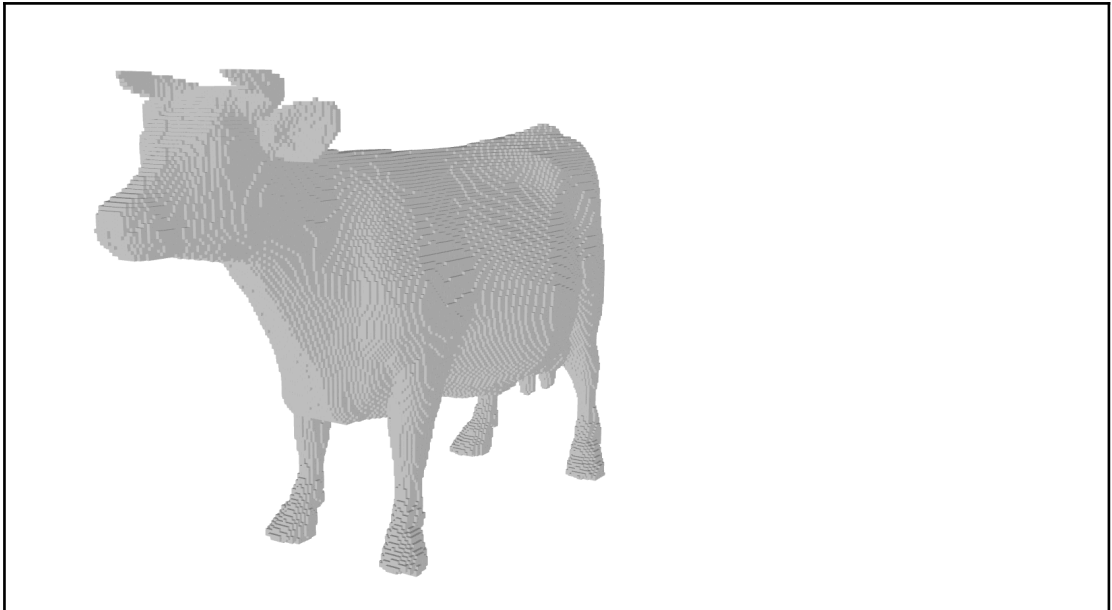
Statistik node yang tidak perlu ditelusuri

1 : 0
2 : 44
3 : 72
4 : 372
5 : 1294
6 : 5269
7 : 22152
8 : 0

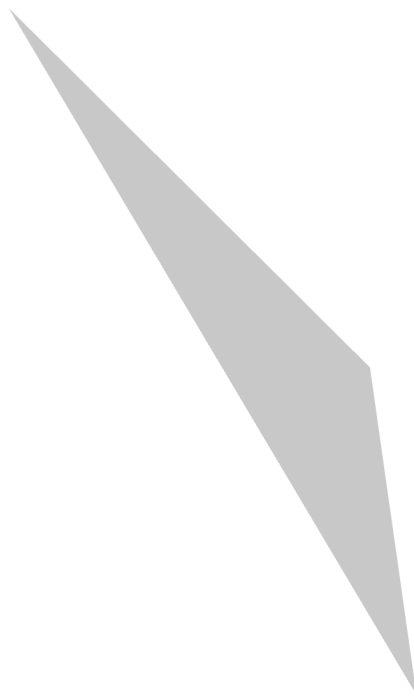
Kedalaman octree: 8

Lama waktu program berjalan: 943.760736ms

Path dimana file .obj disimpan:
test/cow-Voxelization_Result_Depth-8.obj



2. line.obj



Input:

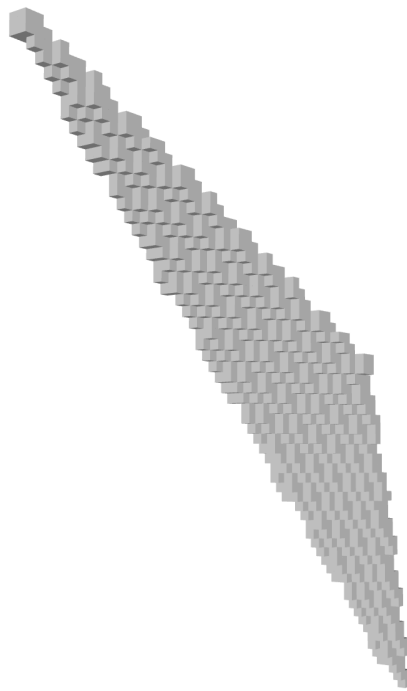
Masukan nama file yang ada di /test (misal : cow.obj) : line.obj
Masukan max depth voxelization Notes : Semakin besar semakin lama waktu menunggu: 6

Output:

Mulai memproses file: test/line.obj
Target kedalaman Octree: 6

=== Statistik Voxelization ===
Banyaknya voxel yang terbentuk: 1018
Banyaknya vertex yang terbentuk: 8144

```
Banyaknya faces yang terbentuk: 12216
Statistik node octree yang terbentuk
1 : 8
2 : 56
3 : 120
4 : 264
5 : 744
6 : 2296
Statistik node yang tidak perlu ditelusuri
1 : 1
2 : 41
3 : 87
4 : 171
5 : 457
6 : 0
Kedalaman octree: 6
Lama waktu program berjalan: 12.975614ms
Path          dimana          file          .obj          disimpan:
test/line-Voxelization_Result_Depth-6.obj
```



```
Input :
Masukan nama file yang ada di /test (misal : cow.obj) : line.obj
Masukan max depth voxelization Notes : Semakin besar semakin lama
waktu menunggu: 8
```

```
Output:
Mulai memproses file: test/line.obj
Target kedalaman Octree: 8

≡≡≡ Statistik Voxelization ≡≡≡
Banyaknya voxel yang terbentuk: 14785
```

Banyaknya vertex yang terbentuk: 118280

Banyaknya faces yang terbentuk: 177420

Statistik node octree yang terbentuk

1 : 8

2 : 56

3 : 120

4 : 264

5 : 744

6 : 2296

7 : 8144

8 : 30600

Statistik node yang tidak perlu ditelusuri

1 : 1

2 : 41

3 : 87

4 : 171

5 : 457

6 : 1278

7 : 4319

8 : 0

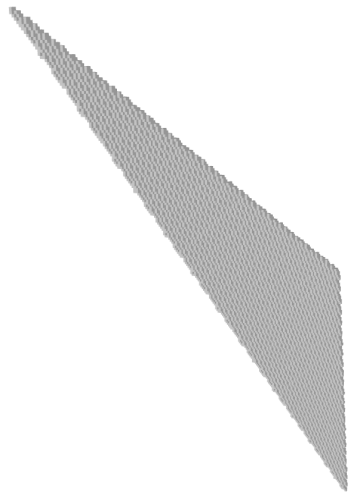
Kedalaman octree: 8

Lama waktu program berjalan: 148.335237ms

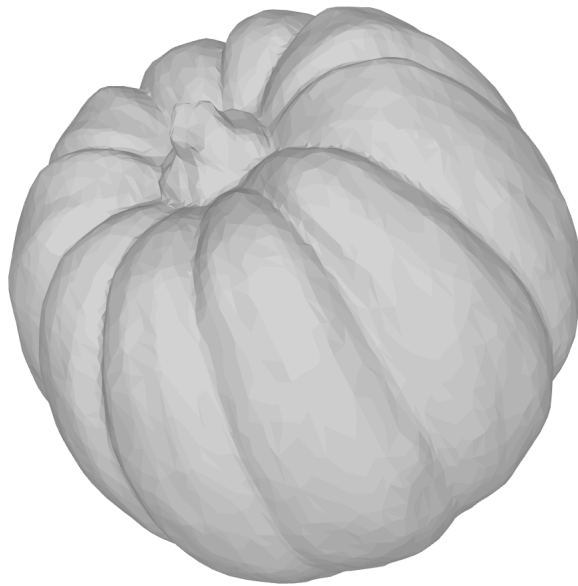
Path dimana file .obj

disimpan:

test/line-Voxelization_Result_Depth-8.obj



3. pumpkin.obj



```
Input : Masukan nama file yang ada di /test (misal : cow.obj) :  
pumpkin.obj  
Masukan max depth voxelization Notes : Semakin besar semakin lama  
waktu menunggu: 6
```

Output:

```
Mulai memproses file: test/pumpkin.obj  
Target kedalaman Octree: 6
```

```
=== Statistik Voxelization ===
```

```
Banyaknya voxel yang terbentuk: 18637
```

```
Banyaknya vertex yang terbentuk: 149096
```

```
Banyaknya faces yang terbentuk: 223644
```

```
Statistik node octree yang terbentuk
```

```
1 : 8
```

```
2 : 64
```

```
3 : 448
```

```
4 : 2216
```

```
5 : 8984
```

```
6 : 37008
```

```
Statistik node yang tidak perlu ditelusuri
```

```
1 : 0
```

```
2 : 8
```

```
3 : 171
```

```
4 : 1093
```

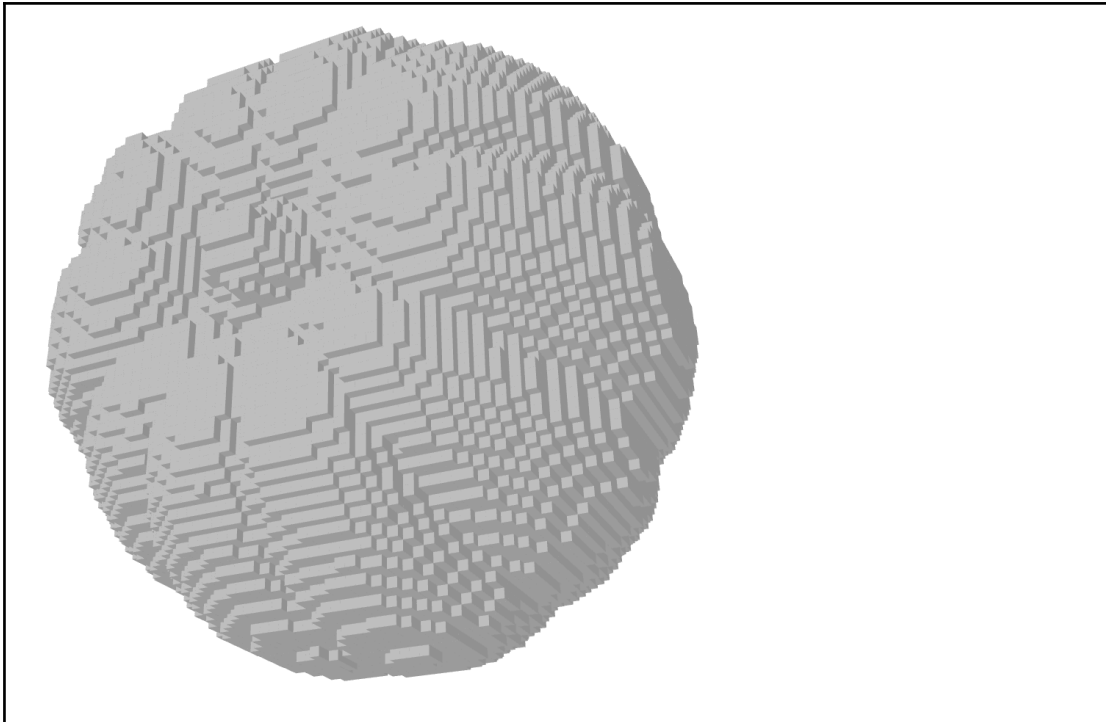
```
5 : 4358
```

```
6 : 0
```

```
Kedalaman octree: 6
```

```
Lama waktu program berjalan: 250.932907ms
```

```
Path          dimana          file          .obj          disimpan:  
test/pumpkin-Voxelization_Result_Depth-6.obj
```

Input :

Masukan nama file yang ada di /test (misal : cow.obj) : pumpkin.obj
Masukan max depth voxelization Notes : Semakin besar semakin lama waktu menunggu: 8

Output:

Mulai memproses file: test/pumpkin.obj
Target kedalaman Octree: 8

=== Statistik Voxelization ===

Banyaknya voxel yang terbentuk: 299111
Banyaknya vertex yang terbentuk: 2392888
Banyaknya faces yang terbentuk: 3589332
Statistik node octree yang terbentuk

1 : 8
2 : 64
3 : 448
4 : 2216
5 : 8984
6 : 37008
7 : 149096
8 : 598000

Statistik node yang tidak perlu ditelusuri

1 : 0
2 : 8
3 : 171
4 : 1093
5 : 4358
6 : 18371
7 : 74346
8 : 0

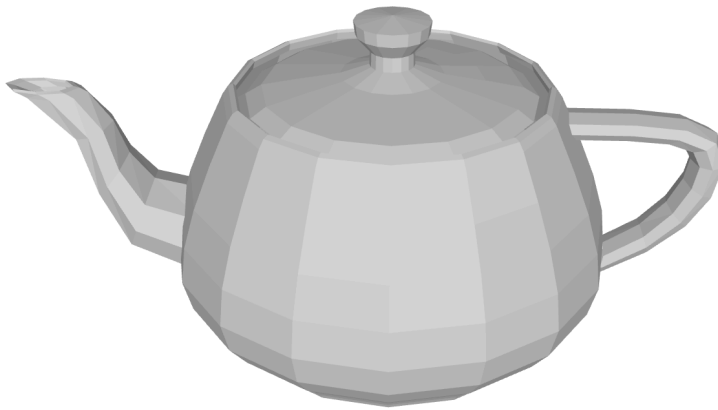
Kedalaman octree: 8

Lama waktu program berjalan: 2.513302005s

Path	dimana	file	.obj	disimpan:
test/pumpkin-Voxelization_Result_Depth-8.obj				



4. teapot.obj



Input :

Masukan nama file yang ada di /test (misal : cow.obj) : teapot.obj
Masukan max depth voxelization Notes : Semakin besar semakin lama waktu menunggu: 6

Output:

Mulai memproses file: test/teapot.obj
Target kedalaman Octree: 6

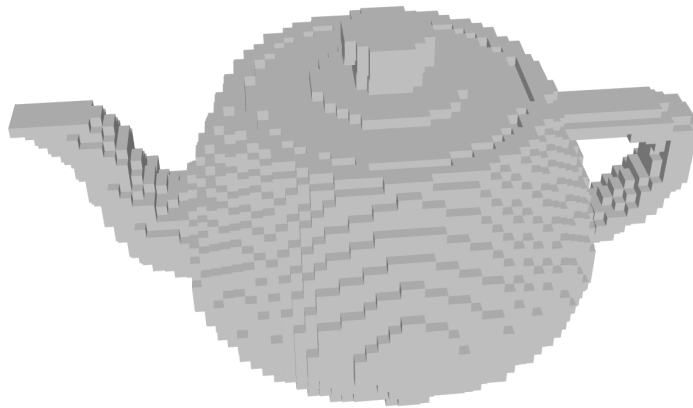
=== Statistik Voxelization ===

Banyaknya voxel yang terbentuk: 7303
Banyaknya vertex yang terbentuk: 58424
Banyaknya faces yang terbentuk: 87636
Statistik node octree yang terbentuk

```

1 : 8
2 : 64
3 : 192
4 : 864
5 : 3344
6 : 13984
Statistik node yang tidak perlu ditelusuri
1 : 0
2 : 40
3 : 84
4 : 446
5 : 1596
6 : 0
Kedalaman octree: 6
Lama waktu program berjalan: 87.505636ms
Path          dimana          file          .obj          disimpan:
test/teapot-Voxelization_Result_Depth-6.obj

```



Input :

Masukan nama file yang ada di /test (misal : cow.obj) : teapot.obj
Masukan max depth voxelization Notes : Semakin besar semakin lama waktu menunggu: 8

Output:

```

Mulai memproses file: test/teapot.obj
Target kedalaman Octree: 8

=== Statistik Voxelization ===
Banyaknya voxel yang terbentuk: 118140
Banyaknya vertex yang terbentuk: 945120
Banyaknya faces yang terbentuk: 1417680
Statistik node octree yang terbentuk
1 : 8
2 : 64
3 : 192
4 : 864
5 : 3344
6 : 13984
7 : 58424
8 : 236520

```

Statistik node yang tidak perlu ditelusuri

1 : 0
2 : 40
3 : 84
4 : 446
5 : 1596
6 : 6681
7 : 28859
8 : 0

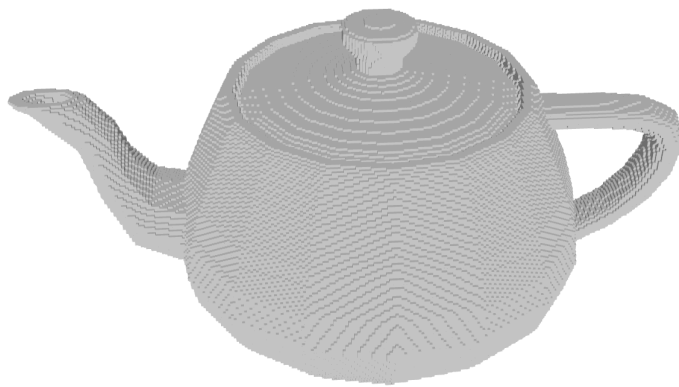
Kedalaman octree: 8

Lama waktu program berjalan: 1.24357235s

Path dimana file .obj

disimpan:

test/teapot-Voxelization_Result_Depth-8.obj



Pengerjaan Bonus

Concurrency memungkinkan program untuk mengeksekusi berbagai tugas secara bersamaan sehingga dapat meningkatkan performa secara drastis. Pada tugas ini, concurrency diimplementasikan dengan merekrut "pekerja" untuk setiap kubus. Namun, perekrutan "pekerja" hanya dilakukan sampai kubus dengan kedalaman 2 saja sehingga maksimal ada 64 proses yang berjalan secara concurrent. Selebihnya untuk kubus dengan kedalaman yang lebih dalam dari itu akan tetap diproses secara sekuensial (satu per satu). Perekrutan "pekerja" hanya dilakukan sampai kedalaman 2 karena mempertimbangkan performa, jumlah "pekerja" akan tumbuh secara eksponensial di setiap kedalamannya dan bekerja secara concurrent sehingga apabila jumlah "pekerja" semakin banyak akan membebani CPU (eksekutor) yang pada akhirnya malah dapat memperlambat performa.

Penerapan concurrency memiliki masalah besar, yaitu race condition. Race condition adalah suatu kondisi ketika ada beberapa proses yang ingin mengakses satu data secara bersamaan. Permasalahan ini dapat memberikan dampak yang sangat fatal, yaitu misinformasi. Sebagai solusinya, kami menerapkan mutex (Mutual Exclusion). Konsep kerja mutex sangatlah sederhana seperti kunci ruangan. Ketika sebuah proses ingin memodifikasi data bersama, maka harus mengunci ruangan tersebut. Jika ada proses lain yang ingin melakukan akses juga namun ruangnya sedang terkunci, maka harus menunggu dan mengantre sampai kunci ruangan terbuka. Sistem antrean ini memastikan bahwa data hanya bisa disentuh satu per satu secara bergantian, sehingga hasil akhirnya tetap aman dan akurat.

Penutup

Berdasarkan implementasi dan pengujian program voxelization 3D yang telah dilakukan, dapat ditarik beberapa kesimpulan utama. Pertama, paradigma *Divide and Conquer* berhasil diimplementasikan secara efektif melalui penggunaan struktur data Octree. Pendekatan ini terbukti menjadi metode yang sangat terstruktur untuk memecah ruang 3D (AABB) secara rekursif menjadi bagian-bagian yang lebih kecil (oktan) untuk mengkonversi polygon kontinu menjadi representasi voxel diskrit. Kedua, terdapat hubungan yang berbanding lurus antara tingkat kedalaman (depth) maksimum Octree dengan tingkat detail visual dari model yang dihasilkan. Semakin tinggi nilai depth yang dimasukkan, ukuran voxel akan semakin kecil dan jumlahnya akan semakin banyak. Hal ini membuat hasil voxelization semakin halus dan mendekati bentuk asli objek 3D. Walaupun begitu, hasil tersebut diperoleh dengan mengorbankan waktu komputasi (runtime) dan ukuran file output yang semakin besar.

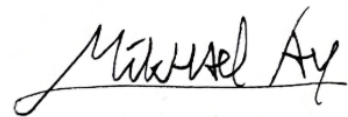
Selanjutnya, Penggunaan Separating Axis Theorem (SAT) berhasil menguji interseksi antara oktan dan permukaan segitiga secara matematis dan akurat. Metode tersebut memungkinkan program melakukan pruning dengan langsung membuang oktan yang kosong tanpa menelusurinya lebih dalam. Hal ini secara drastis meningkatkan efisiensi program karena ruang kosong yang tidak berisi model 3D tidak membebani memori dan komputasi perangkat keras. Terakhir, pada implementasi pada laporan ini telah sukses menghasilkan keluaran file .obj yang valid sesuai spesifikasi, di mana voxel hanya dihasilkan pada bagian permukaan objek (*surface only*).

Lampiran

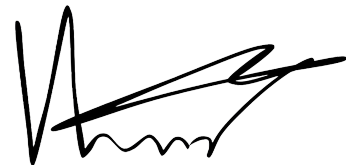
Repository : https://github.com/staplesmaster/Tucil2_13524051_13524105

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✓	
6	[Bonus] Program menggunakan <i>concurrency</i>	✓	
7	[Bonus] Program dapat menampilkan visualisasi .obj		✓

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Mikhael Andrian Yonathan



Nicholas Luis Chandra